

CloudLog — Project Summary

Client-side security log triage & investigation tool · single-file, offline, MITRE ATT&CK-mapped

What it is

CloudLog is a **single-file, offline security log triage tool** — one `index.html` that runs entirely in the browser (no server, no dependencies, nothing leaves the machine). You drop in raw logs, and it flags likely attacks, groups related events into incidents, ranks them by urgency, and helps you investigate.

What it does — the analyst flow

1. Upload a log (auto-detects format)
2. Read plain-English Summary
3. Review MITRE-mapped Detections
4. Investigate ranked P1–P3 incidents
5. Hunt with presets or custom filters

Each incident shows an **entry point**, **attack timeline**, **kill chain**, linked entities, and recommended actions. Plus an evidence graph, optional AI briefing, access codes, and AES-256 encrypted case files. Supports JSON, CSV, syslog, CEF, Apache, Zeek, AWS GuardDuty, Windows/Sysmon, and Splunk/Wazuh/Sentinel exports.

What was built & fixed

Area	Improvement
Investigation	DFIR structure — canonical kill-chain ordering, entry-point identification, attack-timeline reconstruction.
Priority scoring	Fixed "40% everything-is-P1" inflation → meaningful distribution; P1 reserved for confirmed compromise.
Signal vs. noise	Real findings first, verdict-echoes collapsed; added a guarded behavioral layer that stays silent on benign logs.
Windows / Sysmon / Linux	Per-Event-ID readable actions + a Key-fields panel (decoded logon types, command lines, hashes).
Tool security	Fixed a real stored-XSS (attribute quote-escaping); added SECURITY.md + honest crypto/threat model.
Performance	10k-log render freeze cut ~25s → ~5s via lazy tab rendering; fixed a stale detection-cache bug.
Usability	Welcome onboarding, per-tab intros, beginner Hunt presets, sample-replace guard, cramped sidebar forms moved into large modal dialogs.
Code structure	Monolith split into 14 modular <code>src/</code> files + <code>build.js</code> that reassembles byte-identically.

Current state — verified (~250 automated checks, all green)

Build integrity (src ↔ bundle)	✓	End-to-end user journey (7 stages)	clean
Content audit	141/141	Interaction audit	28/28
Investigation methodology	7/7	Windows/Sysmon/Linux mapping	10/10
XSS hardening	6/6	Hunt presets	8/8
Per-rule regression	31/31	Web Worker parity	7/7
Settings modals	7/7	960-case attack corpus	100%

Deliverables

`index.html` — the tool (single-file, deployable)
`src/` (14 modules) + `build.js` — modular source; rebuilds `index.html` byte-identically
`README.md` · `SECURITY.md` · `CHANGELOG.md` · `LICENSE` · `.gitignore`
`docs/` — PROJECT-RECORD, REPO-SETUP, example report, screenshots guide
`tests/` — `rule_regression.js` · `worker_test.js` · `threat-proxy.js` (optional)

Honest limitations

Node-verified, not browser-tested. The logic, data flow, and security are proven by automated tests, but only a human can confirm how it looks and feels on screen — a short browser pass is the recommended final step.

A heuristic triage assistant, not a SIEM replacement. It catches common, known attack patterns and will have false positives/negatives on real data. The browser login is deterrence; the case encryption (AES-256-GCM / PBKDF2) is the strong guarantee. All of this is stated plainly throughout the project.

Built with

Plain HTML/CSS/JavaScript (zero frameworks, zero dependencies), Web Workers, and the Web Crypto API; optional Node threat-intel proxy; hosted via GitHub Pages. Designed and directed by Praharsh, with AI used as a coding assistant — a stance documented openly in the repository.